MediCare Assistant

A grounded medical assistant for clinic booking, pharmacy browsing, and general health information

Ahmad Mourad
Personal Project · 2026

Abstract. MediCare Assistant is a small, production-shaped web application that combines a rule-based task agent with a retrieval-augmented question answering layer. Users can ask for a clinic, book an appointment, check a medicine price, open a demo checkout, or ask a general health question. The agent converts each request into a structured intent and, when the intent is actionable, the browser opens and pre-fills the relevant form. Health answers are grounded in a curated local knowledge base and, when available, a short patient summary from MedlinePlus. The system runs without a large language model by default, and can optionally use an OpenAI-compatible provider. It is implemented with FastAPI, scikit-learn, and vanilla JavaScript, and is covered by automated tests.

1. Introduction

Patients interact with clinic websites through a small number of repeated tasks: find a specialty, book a time, check a price, or ask a simple health question. These tasks are usually spread across separate pages, forms, and search boxes. MediCare Assistant explores a different interaction model: a single conversational entry point that understands the request, explains the answer, and then performs the matching interface action.

The project has four goals:

1. **Grounded answers.** Health information must come from an explicit source, not from an unconstrained model.
2. **Agentic behaviour.** A request such as “book a dentist tomorrow at 5 pm” should fill the booking form, not only return text.
3. **Small operational footprint.** The default deployment should run on a free tier without a GPU, a vector database, or a paid API key.
4. **Security by default.** The demo must not collect payment data, must escape user content, and must apply standard HTTP hardening.

2. System architecture

The application is a single FastAPI service. It serves the static frontend and exposes a JSON API from the same origin, which keeps deployment and CORS configuration simple.

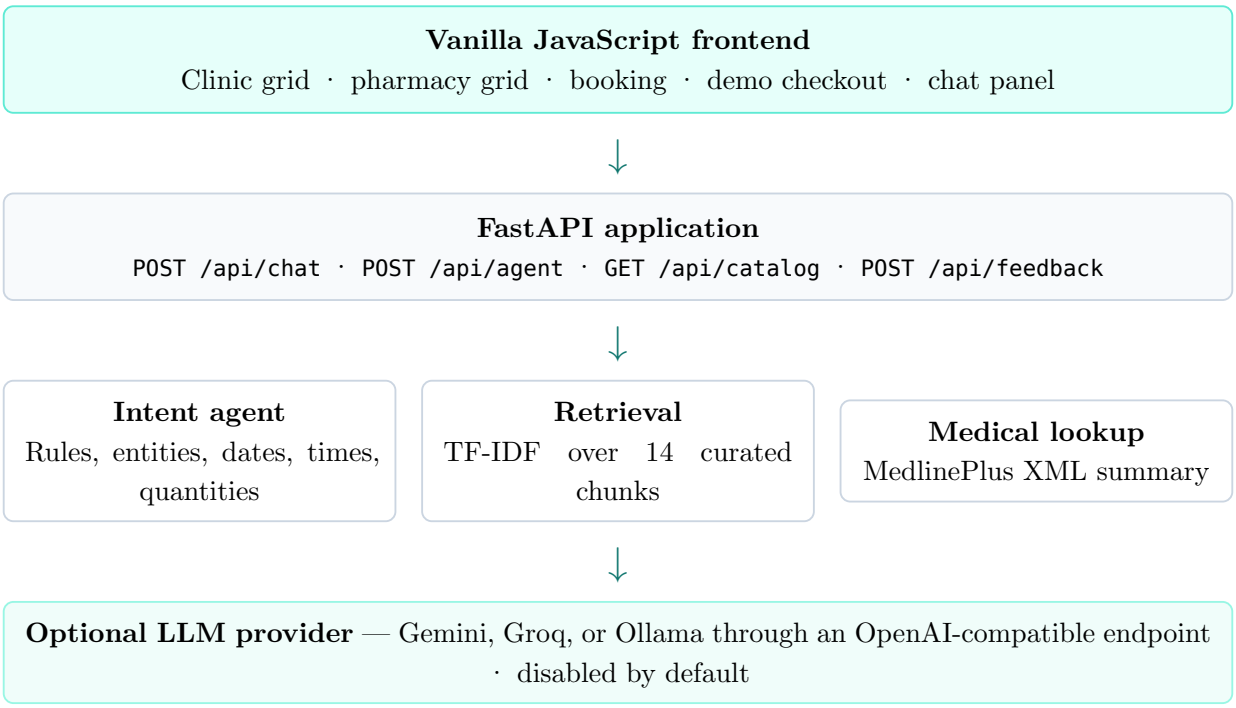


Table 1: Request path. Every request passes through the intent agent. Actionable intents return structured entities to the browser; informational intents use retrieval and optional external grounding.

The frontend is deliberately dependency-free. It renders the catalogue from `Frontend/data.js`, calls the API with `fetch`, and uses DOM APIs rather than injecting HTML strings for user or model output. The backend keeps one cached TF-IDF index in memory. The index is rebuilt only when the process starts or when the knowledge file changes and the cache is cleared.

3. Request flow

A chat request follows five steps:

1. **Normalise.** The message is trimmed, lowercased, and collapsed to single spaces. The API rejects empty messages and messages longer than 500 characters.
2. **Classify.** The agent checks emergency phrases, small talk, medicine and clinic keywords, price and dosage terms, booking terms, purchase terms, and navigation terms. A deterministic priority order prevents ambiguous requests from selecting the wrong action.
3. **Extract.** The agent extracts the clinic, medicine, patient name, quantity, date, and time. Dates accept natural language (“tomorrow”, “next monday”), ISO dates, and day/month names. Times accept 12-hour and 24-hour formats.
4. **Answer.** Actionable intents return a short reply plus an entity object. Informational intents use the retrieval index; medical questions also request a MedlinePlus summary when the feature is enabled. If a provider is configured, the generated answer is constrained by the retrieved context. Otherwise, a deterministic extractive fallback is used and a medical disclaimer is appended.
5. **Act.** The browser reads the returned intent. Booking opens the booking modal with the clinic, name, date, and time filled in. A purchase opens the demo checkout with the medicine, price, and quantity. Navigation intents scroll to the requested section. Price and dosage intents scroll to and highlight the medicine card.

4. Intent coverage

Action	Example	Interface effect
`book_appointment`	book a dentist for Ahmed tomorrow at 5 pm	Booking modal, fields pre-filled
`buy_medicine`	buy two packs of betadine for Ali	Demo checkout, quantity and name pre-filled
`price`	how much is Panadol Extra?	Reply and medicine card highlight
`dosage`	what is the dosage of augmentin?	Reply and medicine card highlight
`medicine_info`	tell me about betadine	Description, price, strength, prescription note
`navigate_clinics`	show me the clinics	Scrolls to the clinics section
`navigate_pharmacy`	open the pharmacy	Scrolls to the pharmacy section
`navigate_contact`	what is your phone number?	Scrolls to contact and lists details
`medical_question`	what is diabetes?	Grounded summary, link, and disclaimer
`emergency`	I have chest pain	Urgent message with local emergency numbers
`unknown`	tell me a joke	Helpful fallback message

Table 2: Supported intents and their visible effect. The same contract is available through `POST /api/agent` for integration tests.

5. Retrieval and grounding

The knowledge base is a Markdown file with headings for clinics, pharmacy, prices, prescriptions, booking rules, shipping, returns, contact details, and assistant capabilities. It is split into heading-aware chunks of at most 600 characters. A TF-IDF vectoriser with unigrams and bigrams is fitted on the chunks at startup. A query is transformed with the same vectoriser and compared with cosine similarity; results with a non-positive score are discarded.

This design was chosen for three reasons. First, it has no model download and no network dependency. Second, it is inspectable: a developer can read the exact context used for an answer. Third, it is fast enough to run on a free instance, with an index of only 14 chunks. The trade-off is lexical matching: synonyms and paraphrases are handled less gracefully than with dense embeddings. The agent compensates with explicit aliases and fuzzy matching for medicine names.

When a message is classified as a medical question, the backend first looks for a MedlinePlus health topic. The lookup uses the NLM web service, parses the XML response, strips HTML entities and tags, and limits the summary to 1,400 characters. The summary is appended to the retrieved context and the source URL is returned to the client. If the lookup fails, the response falls back to the local retrieval result or a safe generic message. Every medical answer ends with a disclaimer that the content is general information and not medical advice.

6. Security and privacy

Control	Implementation
Output encoding	User and model text is inserted with `textContent`; catalogue markup is escaped
Input validation	Pydantic models enforce 1–500 character messages with whitespace normalisation
Rate limiting	Per-IP sliding window, 40 requests per minute by default, disabled with `RATE_LIMIT_PER_MINUTE=0`
Security headers	`nosniff`, `DENY` framing, strict referrer policy, restrictive permissions policy
Content Security Policy	Same-origin scripts, styles, images, and connections; Google Fonts allowed explicitly
Payment data	The demo checkout collects a name and address only; no card, CVV, or expiry fields exist
Secrets	Provider keys are read from environment variables and never committed; `.env` is ignored

Table 3: Application security controls. The documentation routes receive a separate CSP that allows the Swagger UI CDN assets.

The feedback endpoint stores only the message, the predicted action, the corrected action, and a timestamp in a local JSON Lines file. It is intended for offline error analysis and is disabled from the browser interface by default. The file is ignored by Git.

7. Testing and deployment

The repository contains 76 automated tests. They cover intent classification, entity extraction, date and time parsing, fuzzy medicine matching, retrieval ranking, API validation, security headers, rate limiting, feedback persistence, catalogue consistency, local image validity, and the absence of leftover project branding.

Suite	Tests	Focus
`test_agent.py`	45	Actions, entities, dates, times, quantities, safety wording
`test_api.py`	16	Endpoints, validation, headers, rate limit, feedback, docs CSP
`test_retrieval.py`	6	Chunking, ranking, empty index, context formatting
`test_llm.py`	5	Provider gating, fallback formatting, disclaimer
`test_data.py`	4	Catalogue parity, image files, knowledge coverage, branding

Table 4: Test layout. Run `pytest` from the project root after installing `requirements-dev.txt`.

The service is deployed as a Render web service from `render.yaml`. The build command installs `requirements.txt`; the start command is `uvicorn app.main:app --host 0.0.0.0 --port $PORT`; the health check is `/api/health`. The target runtime is Python 3.12. The default configuration uses no LLM provider, which keeps the demo free, deterministic, and reproducible. Supplying `LLM_PROVIDER`, `LLM_API_KEY`, and `LLM_MODEL` enables a hosted or local model without code changes.

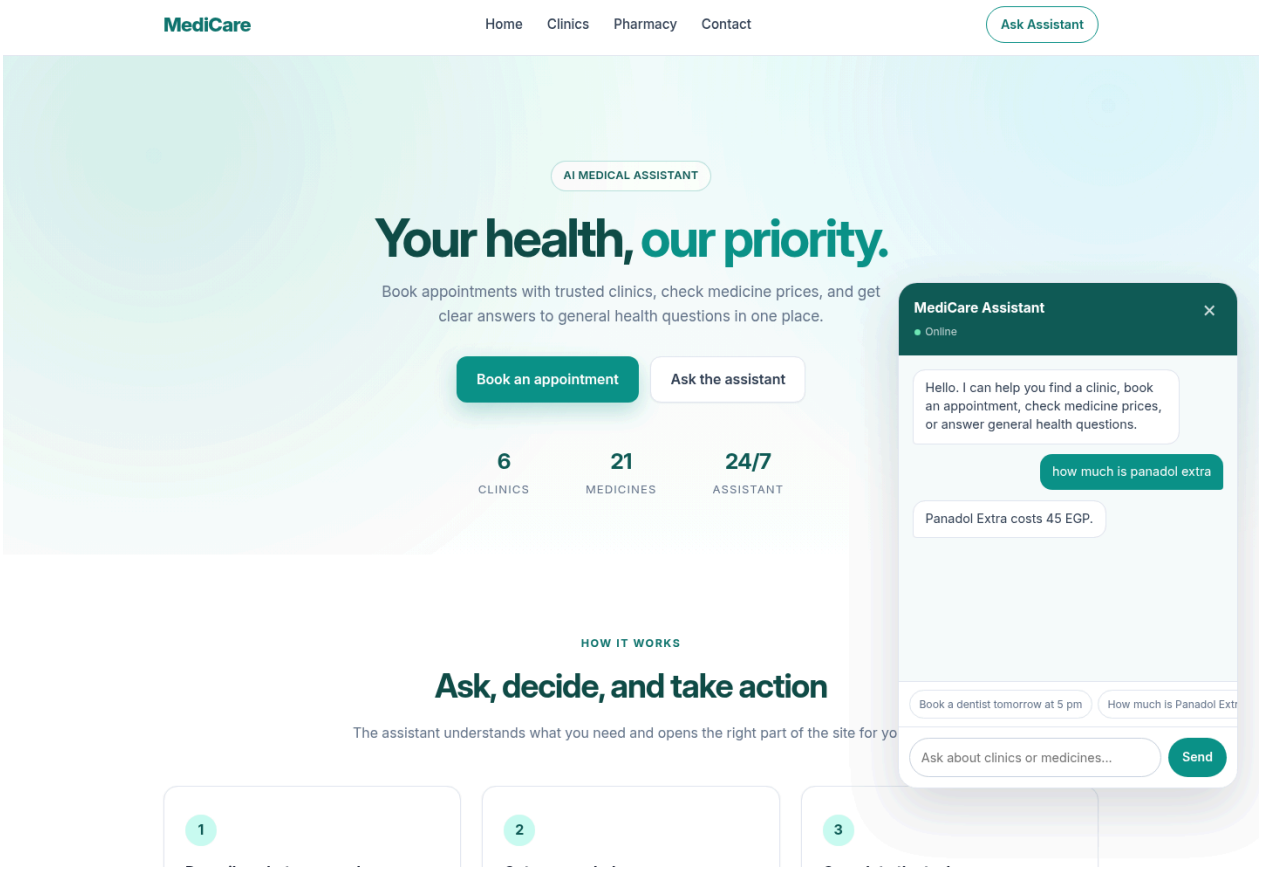


Figure 1: The assistant answers a price question and the pharmacy card is highlighted. The chat panel is the single entry point for all supported tasks.

8. Limitations and future work

The assistant is intentionally narrow. It does not diagnose, prescribe, or provide personalised treatment advice. The medical summary is a short extract from a public reference and may be incomplete or out of date. Clinic availability is demo data; the booking confirmation does not reserve a real slot. The checkout is a demo and does not process payments.

Three extensions are natural:

1. Replace the lexical index with a small dense retriever while keeping the same grounding contract.
2. Add a persistent appointment store and an availability calendar.
3. Add an evaluation set of paraphrased requests to measure intent accuracy and retrieval recall, rather than relying on examples alone.

9. References

1. FastAPI documentation. fastapi.tiangolo.com
2. scikit-learn user guide, text feature extraction. scikit-learn.org
3. MedlinePlus Web Service. medlineplus.gov
4. Typst documentation. typst.app/docs
5. W3C Web Content Accessibility Guidelines. w3.org/WAI